

Universidade Católica de Brasília
Linguagens Formais, Autômatos e Compiladores
2º semestre de 2026

RELATÓRIO TÉCNICO

Analizador Léxico para Assembly MIPS (subconjunto MARS)

Professor: Marcelo Eustáquio

Grupo:

Artur Avallus Cordeiro de Paula

Alexandre Vasconcelos Vicente

Kauã Melo dos Santos

Larissa Carlos de Paulo

1. Introdução

Este relatório documenta a implementação do analisador léxico do subconjunto de Assembly MIPS compatível com o simulador MARS, conforme especificado no enunciado da disciplina. O projeto é dividido em duas etapas encadeadas: o pré-processamento, que remove comentários e normaliza espaços em branco, e a análise léxica propriamente dita, que percorre o código-fonte caractere por caractere, reconhece lexemas, classifica-os em tokens e produz três arquivos de saída: um arquivo de tokens (.lex), a Tabela de Símbolos (.ts) e o relatório de erros léxicos (.err).

A implementação segue a arquitetura de um Autômato Finito Determinístico (AFD), integralmente escrita em linguagem C, sem uso de geradores automáticos de analisadores léxicos (como Lex/Flex), conforme exigido.

O projeto está organizado em quatro módulos: `preprocessador.c/h` (pré-processamento), `lexico.c/h` (análise léxica) e `main.c` (coordenação da execução e leitura dos argumentos de linha de comando).

2. Estruturas de dados

As principais estruturas de dados utilizadas no analisador léxico (arquivo `lexico.c`) são descritas a seguir.

2.1. Reservada

Associa um lexema reservado (instrução ou diretiva) ao nome do token correspondente. É usada para montar as tabelas estáticas `diretivas[]` e `instrucoes[]`, percorridas pela função `buscar()` a cada novo lexema formado por letras.

```
typedef struct {
    const char *lexema;
    const char *nome_token;
} Reservada;
```

2.2. Simbolo

Representa uma entrada da Tabela de Símbolos, correspondendo à estrutura sugerida no enunciado (campos `lexema`, `categoria`, `linha` e `coluna` da primeira ocorrência).

```
typedef struct {
    char lexema[MAX_LEXEMA];
    char categoria[32];
    int linha;
    int coluna;
} Simbolo;
```

2.3. Tabelas estáticas de palavras reservadas

São três vetores terminados por uma entrada `{ NULL, NULL }` (ou `NULL`, no caso dos registradores), usados tanto para reconhecer os lexemas reservados quanto para pré-carregar a Tabela de Símbolos:

- `diretivas[]` — as 10 diretivas do subconjunto (`.data`, `.text`, `.word`, `.half`, `.byte`, `.space`, `.ascii`, `.asciiz`, `.globl`, `.align`);

- `instrucoes[]` — as 36 instruções/pseudoinstruções do subconjunto (transferência de dados, aritméticas, lógicas/deslocamento/comparação, desvio e controle);
- `registradores_nomeados[]` — os 31 nomes simbólicos de registradores (as formas numéricas \$0 a \$31 são validadas por cálculo, não por tabela).

2.4. Constantes

Constante	Valor	Finalidade
MAX_LEXEMA	4096	Tamanho máximo de um lexema individual (string, identificador, etc.)
MAX_SIMBOLOS	2048	Capacidade máxima da Tabela de Símbolos
TAM_LINHA	4096	Tamanho do buffer de leitura de uma linha (fgets) no pré-processador
TAM_LINHA_ACUMULADA	TAM_LINHA * 4 = 16384	Buffer para acumular fragmentos de uma linha maior que TAM_LINHA

3. Funções implementadas

3.1. Módulo de pré-processamento (preprocessador.c)

Função	Responsabilidade
<code>remover_comentario</code>	Remove tudo a partir de '#' até o fim da linha, exceto quando o '#' está dentro de uma cadeia de caracteres (delimitada por aspas) fechada na mesma linha.
<code>normalizar_espacos</code>	Colapsa espaços/tabulações consecutivos em um único espaço fora de strings, preserva o conteúdo exato dentro de strings e remove \r e \n do resultado.
<code>processar_linha</code>	Aplica <code>remover_comentario</code> e <code>normalizar_espacos</code> a uma linha e grava o resultado no arquivo de saída, descartando linhas que ficaram vazias.
<code>preprocessar</code>	Lê o arquivo de entrada, reagrupando fragmentos de uma mesma linha física que excedam TAM_LINHA (uma linha muito longa é lida em pedaços por fgets); chama <code>processar_linha</code> para cada linha completa.
<code>PreProcessamento</code>	Interface exigida literalmente pelo enunciado (void PreProcessamento(FILE *in, FILE *out)); encaminha para <code>preprocessar()</code> , mantendo a implementação concentrada em uma única função.

3.2. Módulo de análise léxica (lexico.c)

Função	Responsabilidade
<code>buscar</code>	Procura um lexema (já normalizado para minúsculas) nas tabelas <code>diretivas[]</code> ou <code>instrucoes[]</code> , retornando o nome do token ou NULL.
<code>eh_registrador</code>	Verifica se um lexema iniciado por '\$' corresponde a um dos 31 nomes simbólicos ou a uma forma numérica \$0..\$31.
<code>fim_lexema</code>	Define os delimitadores que encerram a leitura de um lexema: espaço em branco, ',', ':', '(', ')', '#' ou fim de arquivo.

Função	Responsabilidade
para_minusculas	Copia um lexema convertendo cada caractere para minúsculo; usada para normalizar instruções, diretivas e registradores antes da busca e do registro, sem afetar rótulos/identificadores (sensíveis a maiúsculas/minúsculas).
identificador_valido	Valida a gramática (letra _)(letra dígito _)* sobre o lexema já formado, detectando identificadores malformados como soma-total ou inicio@.
numero_valido	Valida um lexema numérico completo (decimal ou hexadecimal, com sinal negativo opcional), usada após o lexema já ter sido lido até o delimitador.
registrar_simbolo	Insere um novo símbolo na Tabela de Símbolos (uma única vez por lexema) ou, se o símbolo já existir pré-carregado com posição (0,0), atualiza para a posição da primeira ocorrência real.
emitir	Grava uma linha no arquivo .lex no formato <TOKEN, lexema> linha coluna.
registrar_erro	Grava uma linha no arquivo .err no formato <TIPO_ERRO, lexema> linha coluna.
AnaliseLexicaComSaidas	Função principal do AFD: percorre a entrada caractere por caractere, despachando para o bloco de reconhecimento correspondente ao estado inicial detectado, e escreve os três arquivos de saída ao final.
AnaliseLexica	Interface exigida literalmente pelo enunciado (void AnaliseLexica(FILE *in, FILE *out)); delega para AnaliseLexicaComSaidas usando arquivos temporários para a TS e os erros. O main.c do projeto usa diretamente AnaliseLexicaComSaidas para poder gravar .ts e .err em arquivos reais.

3.3. Módulo principal (main.c)

Função	Responsabilidade
nome_relatorio	A partir do nome do arquivo .lex informado na linha de comando, deriva os nomes dos arquivos .ts e .err trocando a extensão.
main	Valida os 3 argumentos obrigatórios, abre os arquivos, chama PreProcessamento(entrada, saida_pre) e, em seguida, AnaliseLexicaComSaidas(saida_pre, saida_lex, saida_ts, saida_err); trata erros de abertura de arquivo com mensagens claras e encerramento controlado.

4. Autômato Finito Determinístico (AFD)

O diagrama a seguir representa o AFD idealizado do analisador léxico, cobrindo todos os caminhos exigidos pelo enunciado: identificadores/instruções, diretivas, registradores, números decimais/hexadecimais/negativos, cadeias de caracteres (com estado de escape), comentários, os símbolos de um único caractere e o estado de erro.

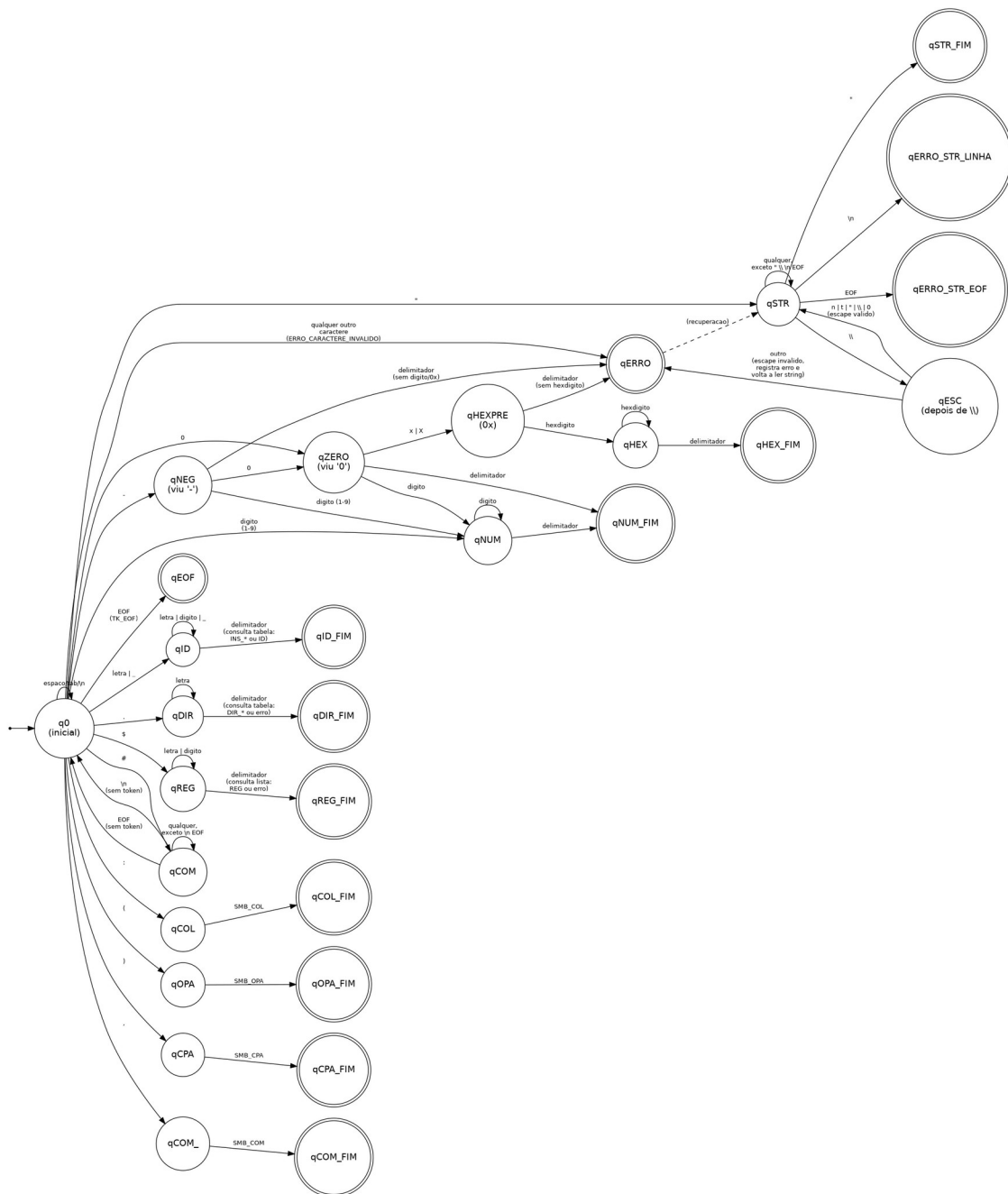


Figura 1 — AFD do analisador léxico (estados de aceitação em círculo duplo).

O estado inicial (q0) despacha a leitura conforme o primeiro caractere do lexema: letra/'_' leva ao estado de identificador/instrução (qID); '.' ao estado de diretiva (qDIR); '\$' ao estado de registrador (qREG); dígito ou '-' ao estado de número (qNUM/qZERO/qNEG); '"' ao estado de cadeia (qSTR); '#' ao estado de comentário (qCOM); os símbolos de um único caractere (';', ':', '(', ')') levam diretamente a um estado final; qualquer outro caractere leva ao estado de erro (qERRO).

Uma diferença consciente entre o diagrama e a implementação merece registro: no diagrama, cada estado avança apenas com o caractere que de fato pertence ao padrão do token (por exemplo, qID só continua com letra, dígito ou '_'). Na implementação em lexico.c, por simplicidade e para permitir uma mensagem de erro com o lexema completo, os estados de identificador, diretiva, registrador e número consomem todos os caracteres até encontrar um delimitador

(função `fim_lexema`) e só então validam o lexema completo (funções `identificador_valido`, `buscar`, `eh_registrador` e `numero_valido`). Essa estratégia é explicitamente admitida pelo enunciado ("uma possível organização dos estados é...") e é o que permite, por exemplo, que um lexema malformado como `soma-total` ou `0xG1` seja reportado por inteiro no arquivo `.err`, em vez de fragmentado em múltiplos erros de um caractere só.

5. Estratégia de controle de linha e coluna

As variáveis `linha` e `coluna` são inicializadas em 1 e atualizadas a cada caractere lido de entrada:

- Ao ler `'\n'`, `linha` é incrementada e `coluna` volta a 1;
- Ao ler `'\r'` (quebra de linha estilo Windows, CRLF), o caractere é descartado sem alterar `linha` nem `coluna` — dessa forma, o mesmo arquivo processado com CRLF ou com LF puro produz exatamente a mesma saída (`.lex`, `.ts` e `.err` idênticos, byte a byte, conforme verificado nos testes);
- Ao ler espaço ou tabulação, `coluna` é incrementada e nenhum token é gerado;
- Para cada lexema, `linha_inicio` e `coluna_inicio` são capturados antes de iniciar a leitura do lexema, e são esses valores — não a posição final — que são gravados nos arquivos `.lex` e `.err`, conforme exigido;
- Ao atingir o fim do arquivo, o token especial `TK_EOF` é emitido com a posição (`linha`, `coluna`) alcançada após o último caractere lido.

Como os lexemas de identificador/instrução, diretiva, registrador e número terminam ao encontrar um delimitador (e não um caractere que já faz parte do próximo token), a implementação usa `ungetc()` para devolver esse caractere delimitador ao fluxo de entrada antes de continuar o laço principal, garantindo que ele seja reprocessado normalmente na iteração seguinte.

6. Estratégia de tratamento e recuperação de erros

O analisador nunca interrompe a execução ao encontrar um erro léxico: cada erro é registrado no arquivo `.err` (tipo, lexema/caractere, linha e coluna) e a análise continua a partir do próximo caractere não consumido, de modo que o arquivo `.err` reúne todos os erros do programa, e não apenas o primeiro. Os oito tipos mínimos exigidos pelo enunciado foram implementados:

Tipo de erro	Situação detectada
ERRO_CARACTERE_INVALIDO	Caractere que não inicia nenhum estado válido do AFD (ex.: <code>@</code>).
ERRO_IDENTIFICADOR_MALFORMADO	Lexema iniciado por letra/'_' que contém caractere fora de <code>(letra dígito _)</code> , ex.: <code>soma-total</code> , <code>inicio@</code> .
ERRO_NUMERO_MALFORMADO	Lexema numérico que não corresponde a decimal, decimal negativo, hexadecimal ou hexadecimal negativo válido, ex.: <code>0xG1</code> .
ERRO_DIRETIVA_INVALIDA	Lexema iniciado por <code>'.'</code> que não pertence às 10 diretivas do subconjunto, ex.: <code>.dados</code> .
ERRO_REGISTRADOR_INVALIDO	Lexema iniciado por <code>'\$'</code> que não corresponde a um registrador válido, ex.: <code>\$t10</code> .

Tipo de erro	Situação detectada
ERRO_STRING_NAO_FECHADA_LINHA	Cadeia de caracteres não fechada antes de uma quebra de linha.
ERRO_STRING_NAO_FECHADA_EOF	Cadeia de caracteres não fechada antes do fim do arquivo.
ERRO_SEQUENCIA_ESCAPE_INVALIDA	Sequência de escape dentro de uma string diferente de \n, \t, \", \\ ou \0.

Observação sobre o par ERRO_STRING_NAO_FECHADA_LINHA / ERRO_STRING_NAO_FECHADA_EOF: como o pré-processador grava cada linha processada sempre finalizada por '\n' (mesmo quando o arquivo original não termina em quebra de linha), o caso _EOF praticamente só é observável quando o léxico é alimentado diretamente por um arquivo sem passar pelo pré-processamento. Isso foi verificado experimentalmente: o mesmo arquivo erro3.asm (que termina sem '\n') produz ERRO_STRING_NAO_FECHADA_EOF quando lido diretamente pelo léxico e ERRO_STRING_NAO_FECHADA_LINHA quando processado pelo pipeline completo (main.exe), porque o pré-processador já insere a quebra de linha final antes de entregar o texto ao léxico. Ambos os tipos de erro estão implementados e a distinção continua válida — a diferença observada é um efeito colateral esperado da normalização feita na etapa anterior.

No caso específico de uma sequência de escape inválida, a análise não interrompe a leitura da string: o erro é registrado e a leitura continua normalmente até a aspa de fechamento (ou até um novo erro de string não fechada), permitindo detectar múltiplos problemas na mesma cadeia.

7. Tabela de Símbolos

A Tabela de Símbolos é pré-carregada, antes de iniciar a leitura do código-fonte, com todas as diretivas, instruções e registradores previstos no subconjunto adotado, todos com posição inicial (0, 0). Durante a análise, cada vez que um desses símbolos reservados é efetivamente utilizado no programa, sua posição é atualizada para a linha e coluna da primeira ocorrência real (a função registrar_simbolo só atualiza a posição de um símbolo que ainda está em (0, 0); usos subsequentes do mesmo símbolo não alteram mais a posição registrada).

Identificadores usados como rótulos são adicionados à tabela na primeira vez em que aparecem (seja em uma declaração, como laco:, seja em uma referência antecipada, como j laco), com a categoria "identificador/rotulo". Rótulos são sensíveis a maiúsculas e minúsculas (inicio e Inicio são símbolos distintos), enquanto instruções, diretivas e registradores são normalizados para minúsculas antes de serem comparados e armazenados (por exemplo, ADD, \$T0 e .TEXT são reconhecidos e gravados na tabela como add, \$t0 e .text).

Conforme exigido, números, cadeias de caracteres e símbolos de pontuação (vírgula, dois-pontos, parênteses) nunca são inseridos na Tabela de Símbolos — apenas os tokens NUM_INT, STRING e SMB_* são emitidos no arquivo .lex, sem passar por registrar_simbolo. Cada lexema aparece uma única vez na tabela, independentemente de quantas vezes seja utilizado no código.

8. Testes realizados

Foram elaborados 10 programas de teste, cobrindo todas as categorias mínimas exigidas pelo enunciado, executados de ponta a ponta através do pipeline completo (main.exe entrada.asm saída.pre saída.lex):

Arquivo	Cobertura
correto1.asm, correto2.asm, correto3.asm	Três programas lexicalmente corretos, sem nenhum erro; correto3.asm é o mais completo, combinando diretivas de dados, rótulos, instruções aritméticas e de desvio, e endereçamento com deslocamento.
erro1.asm	Registrador inválido (\$t10, fora do intervalo \$t0-\$t9).
erro2.asm	Caractere inválido (@) isolado no meio de uma instrução.
erro3.asm	Cadeia de caracteres não fechada (arquivo termina no meio de uma string).
comentarios.asm	Comentário iniciado por '#', inclusive um '#' dentro de uma string (que deve ser preservado, não tratado como início de comentário).
strings.asm	Cadeias de caracteres com sequências de escape válidas (\n e \").
hexadecimal.asm	Números em formato decimal e hexadecimal.
memoria.asm	Acesso à memória no formato deslocamento(registrador), ex.: 4(\$sp).

Além dos testes com arquivos .asm, foram feitas as seguintes verificações adicionais durante o desenvolvimento:

- Compilação com gcc -std=c99 -Wall -Wextra -pedantic, sem nenhum warning, para os três módulos (main.c, preprocessador.c, lexico.c);
- Verificação com AddressSanitizer (-fsanitize=address,undefined) para detectar acessos inválidos de memória, incluindo um teste com uma linha de 5000 caracteres para validar a correção de um estouro de buffer identificado no pré-processador;
- Comparação byte a byte das saídas (.lex, .ts, .err) do mesmo conteúdo processado com quebras de linha estilo Windows (CRLF) e estilo Unix (LF), confirmando saída idêntica;
- Testes unitários do pré-processador (test_preprocessador.c), cobrindo remoção de comentários, preservação de '#' dentro de strings, normalização de espaços/tabulações, remoção de linhas vazias e preservação de linha longa — todos aprovados.

9. Saídas geradas pelo analisador

9.1. Exemplo completo — correto3.asm

Programa de entrada:

```
.data
valor: .word 10
texto: .ascii "Teste"

.text
```

```

main:
li $t0, 10
addi $t1, $t0, -5
lw $t2, 4($sp)
sw $t2, 8($sp)
beq $t0, $t1, fim
j fim

fim:
nop

```

Saída gerada em correto3.lex:

```

<DIR_DATA, .data> 1 1
<ID, valor> 2 1
<SMB_COL, :> 2 6
<DIR_WORD, .word> 2 8
<NUM_INT, 10> 2 14
<ID, texto> 3 1
<SMB_COL, :> 3 6
<DIR_ASCII, .ascii> 3 8
<STRING, "Teste"> 3 15
<DIR_TEXT, .text> 4 1
<ID, main> 5 1
<SMB_COL, :> 5 5
<INS_LI, li> 6 1
<REG, $t0> 6 4
<SMB_COM, ,> 6 7
<NUM_INT, 10> 6 9
<INS_ADDI, addi> 7 1
<REG, $t1> 7 6
<SMB_COM, ,> 7 9
<REG, $t0> 7 11
<SMB_COM, ,> 7 14
<NUM_INT, -5> 7 16
<INS_LW, lw> 8 1
<REG, $t2> 8 4
<SMB_COM, ,> 8 7
<NUM_INT, 4> 8 9
<SMB_OPA, (> 8 10
<REG, $sp> 8 11
<SMB_CPA, )> 8 14
<INS_SW, sw> 9 1
<REG, $t2> 9 4
<SMB_COM, ,> 9 7
<NUM_INT, 8> 9 9
<SMB_OPA, (> 9 10
<REG, $sp> 9 11
<SMB_CPA, )> 9 14
<INS_BEQ, beq> 10 1
<REG, $t0> 10 5
<SMB_COM, ,> 10 8
<REG, $t1> 10 10
<SMB_COM, ,> 10 13
<ID, fim> 10 15
<INS_J, j> 11 1
<ID, fim> 11 3
<ID, fim> 12 1
<SMB_COL, :> 12 4
<INS_NOP, nop> 13 1
<TK_EOF, EOF> 14 1

```

Saída gerada em correto3.err (nenhum erro encontrado):

Nenhum erro lexico encontrado.

9.2. Exemplos de erros — erro1.asm, erro2.asm e erro3.asm

erro1.asm (linha 3: li \$t10, 5) produz em erro1.err:

```
<ERRO_REGISTRADOR_INVALIDO, $t10> 3 4
```

erro2.asm (linha 3: li \$t0, 5 @) produz em erro2.err:

```
<ERRO_CARACTERE_INVALIDO, @> 3 11
```

erro3.asm (arquivo termina em .asciiz "Mensagem, sem fechar a string) produz em erro3.err:

```
<ERRO_STRING_NAO_FECHADA_LINHA, "Mensagem"> 2 14
```

9.3. Cadeias de caracteres com escape — strings.asm

```
<DIR_DATA, .data> 1 1
<ID, msg1> 2 1
<SMB_COL, :> 2 5
<DIR_ASCIIZ, .asciiz> 2 7
<STRING, "Ola mundo"> 2 15
<ID, msg2> 3 1
<SMB_COL, :> 3 5
<DIR_ASCIIZ, .asciiz> 3 7
<STRING, "Linha 1\nLinha 2"> 3 15
<ID, msg3> 4 1
<SMB_COL, :> 4 5
<DIR_ASCIIZ, .asciiz> 4 7
<STRING, "Aspas: \"teste\""> 4 15
<TK_EOF, EOF> 5 1
```

9.4. Endereçamento deslocamento(registrador) — memoria.asm

Entrada: lw \$t0, 4(\$sp)

```
<INS_LW, lw> 1 1
<REG, $t0> 1 4
<SMB_COM, ,> 1 7
<NUM_INT, 4> 1 9
<SMB_OPA, (> 1 10
<REG, $sp> 1 11
<SMB_CPA, )> 1 14
<TK_EOF, EOF> 2 1
```

10. Conteúdo final da Tabela de Símbolos

A seguir, o conteúdo completo do arquivo correto3.ts, gerado a partir do programa da seção 9.1. Observe que todas as instruções, diretivas e registradores do subconjunto aparecem desde a inicialização (posição 0,0 quando não utilizados no programa), e que os símbolos efetivamente usados (.data, .text, .word, .ascii, addi, beq, j, li, lw, sw, nop, os registradores \$t0/\$t1/\$t2/\$sp e os rótulos valor/texto/main/fim) aparecem com a posição real da primeira ocorrência.

```
LEXEMA, CATEGORIA, LINHA, COLUNA
.data, diretiva, 1, 1
.text, diretiva, 4, 1
.word, diretiva, 2, 8
.asciiz, diretiva, 0, 0
```

```
.ascii,diretiva,3,8
.half,diretiva,0,0
.byte,diretiva,0,0
.space,diretiva,0,0
.globl,diretiva,0,0
.align,diretiva,0,0
add,instrucao,0,0
addi,instrucao,7,1
addu,instrucao,0,0
and,instrucao,0,0
beq,instrucao,10,1
bne,instrucao,0,0
blt,instrucao,0,0
ble,instrucao,0,0
bgt,instrucao,0,0
bge,instrucao,0,0
div,instrucao,0,0
j,instrucao,11,1
jal,instrucao,0,0
jr,instrucao,0,0
li,instrucao,6,1
lw,instrucao,8,1
la,instrucao,0,0
sw,instrucao,9,1
lb,instrucao,0,0
sb,instrucao,0,0
lh,instrucao,0,0
sh,instrucao,0,0
move,instrucao,0,0
mul,instrucao,0,0
nor,instrucao,0,0
or,instrucao,0,0
sll,instrucao,0,0
slt,instrucao,0,0
srl,instrucao,0,0
sub,instrucao,0,0
subu,instrucao,0,0
mflo,instrucao,0,0
mfhi,instrucao,0,0
syscall,instrucao,0,0
xor,instrucao,0,0
nop,instrucao,13,1
$zero,registrador,0,0
$v0,registrador,0,0
$v1,registrador,0,0
$a0,registrador,0,0
$a1,registrador,0,0
$a2,registrador,0,0
$a3,registrador,0,0
$t0,registrador,6,4
$t1,registrador,7,6
$t2,registrador,8,4
$t3,registrador,0,0
$t4,registrador,0,0
$t5,registrador,0,0
$t6,registrador,0,0
$t7,registrador,0,0
$t8,registrador,0,0
$t9,registrador,0,0
$s0,registrador,0,0
$s1,registrador,0,0
$s2,registrador,0,0
$s3,registrador,0,0
$s4,registrador,0,0
$s5,registrador,0,0
$s6,registrador,0,0
$s7,registrador,0,0
```

```
$k0,registrador,0,0  
$k1,registrador,0,0  
$gp,registrador,0,0  
$sp,registrador,8,11  
$fp,registrador,0,0  
$ra,registrador,0,0  
valor,identificador/rotulo,2,1  
texto,identificador/rotulo,3,1  
main,identificador/rotulo,5,1  
fim,identificador/rotulo,10,15
```

11. Considerações finais

A implementação atende a todos os requisitos funcionais especificados: reconhecimento completo do subconjunto de tokens (diretivas, instruções, identificadores, registradores, números, cadeias e símbolos), controle de linha e coluna, Tabela de Símbolos pré-carregada e normalizada, detecção e recuperação dos oito tipos mínimos de erro léxico, e geração dos três arquivos de saída exigidos. O código foi validado com compilação sem warnings (-Wall -Wextra -pedantic), análise dinâmica com AddressSanitizer e um conjunto de 10 casos de teste cobrindo todas as categorias mínimas exigidas pelo enunciado.